

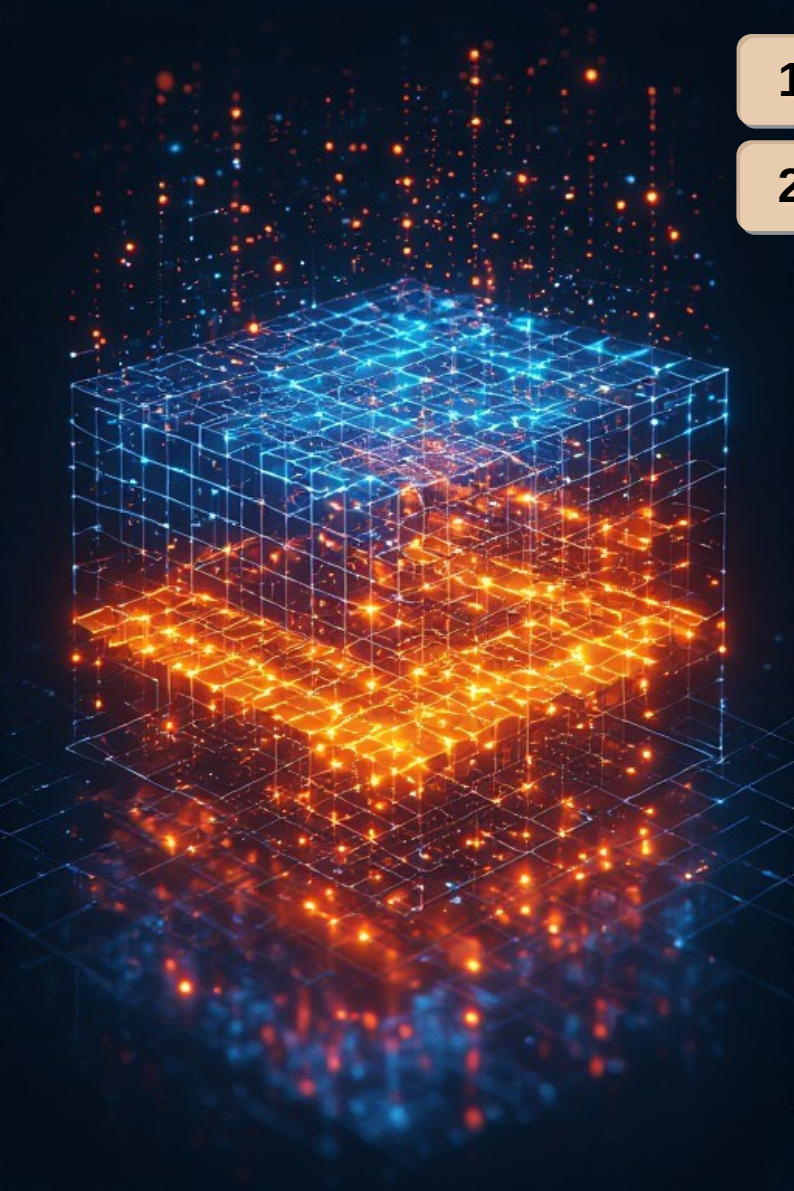


Universidad Nacional de La Plata



Facultad de Informática





1

**Multiplicación de matrices y organización en memoria**

2

**Solución CUDA en GPU Nvidia**

**1**

## **Multiplicación de matrices y organización en memoria**



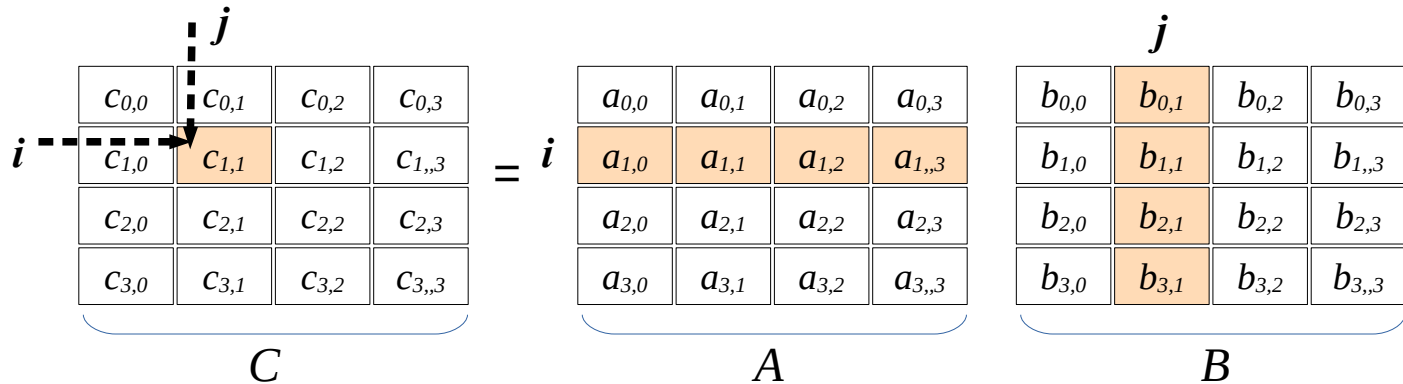
# Multiplicación de matrices

4

- Dadas dos matrices  $A$  y  $B$  de  $N \times N$  elementos. La multiplicación  $AB$  retorna una matriz resultado  $C$  donde cada elemento se calcula como:

$$c_{i,j} = \sum_{k=0}^{N-1} a_{i,k} \cdot b_{k,j}$$

- Para calcular la posición  $(i,j)$  de la matriz resultante  $C$  es necesario procesar la fila  $i$  de  $A$  y la columna  $j$  de  $B$ .

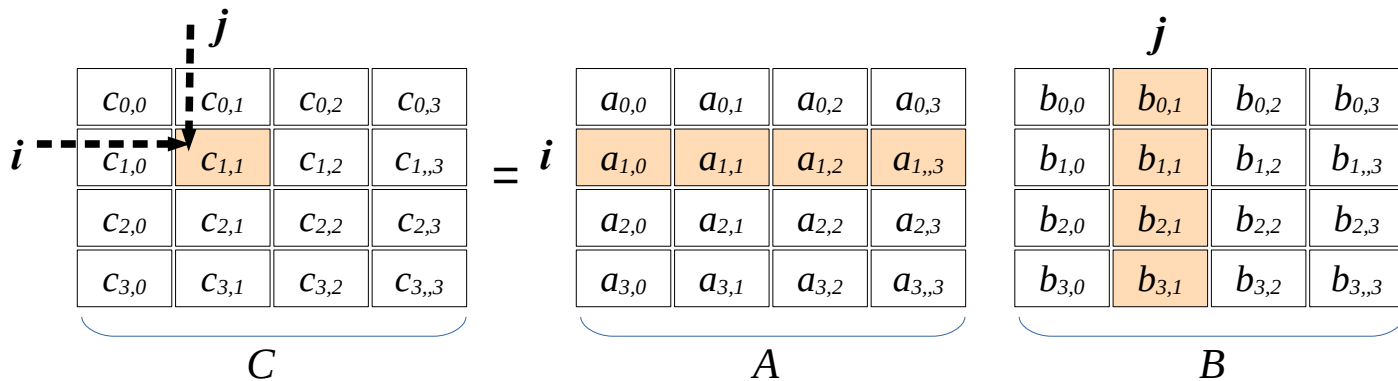




# Organización de matrices en memoria

5

- Al igual que en arquitecturas paralelas tradicionales y por razones de localidad, las matrices en GPU también se tratan como arreglos ordenados en memoria de acuerdo al patrón en el cual se acceden:



Por filas:  $A$ 

|           |           |           |           |           |           |           |           |           |           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $a_{0,0}$ | $a_{0,1}$ | $a_{0,2}$ | $a_{0,3}$ | $a_{1,0}$ | $a_{1,1}$ | $a_{1,2}$ | $a_{1,3}$ | $a_{2,0}$ | $a_{2,1}$ | $a_{2,2}$ | $a_{2,3}$ | $a_{3,0}$ | $a_{3,1}$ | $a_{3,2}$ | $a_{3,3}$ |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|

Por columnas:  $B$ 

|           |           |           |           |           |           |           |           |           |           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $b_{0,0}$ | $b_{1,0}$ | $b_{2,0}$ | $b_{3,0}$ | $b_{0,1}$ | $b_{1,1}$ | $b_{2,1}$ | $b_{3,1}$ | $b_{0,2}$ | $b_{1,2}$ | $b_{2,2}$ | $b_{3,2}$ | $b_{0,3}$ | $b_{1,3}$ | $b_{2,3}$ | $b_{3,3}$ |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|

Por filas:  $C$ 

|           |           |           |           |           |           |           |           |           |           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $c_{0,0}$ | $c_{0,1}$ | $c_{0,2}$ | $c_{0,3}$ | $c_{1,0}$ | $c_{1,1}$ | $c_{1,2}$ | $c_{1,3}$ | $c_{2,0}$ | $c_{2,1}$ | $c_{2,2}$ | $c_{2,3}$ | $c_{3,0}$ | $c_{3,1}$ | $c_{3,2}$ | $c_{3,3}$ |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|

Acceso a la posición  $(i,j)$  de una matriz  $M$  ordenada por filas:  $M[i * N + j]$

Acceso a la posición  $(i,j)$  de una matriz  $M$  ordenada por columnas:  $M[i + j * N]$

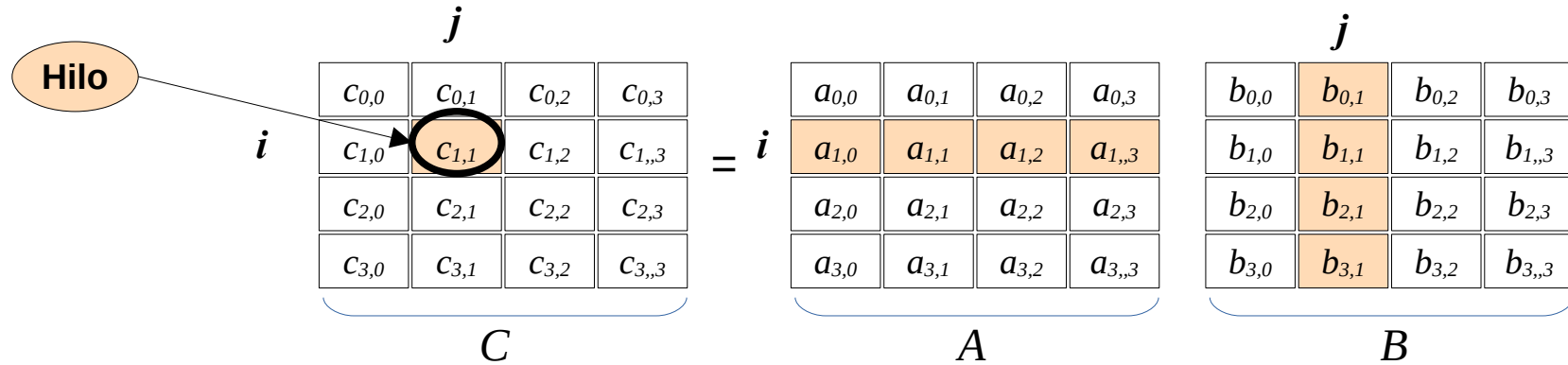


# Multiplicación de matrices

## Solución en GPU

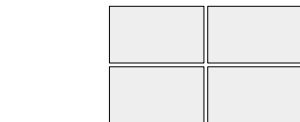
7

- La solución en GPU asigna a cada hilo el cálculo de una celda de la matriz resultado.
- Cada hilo CUDA necesita acceder a la fila  $i$  de  $A$  y la columna  $j$  de  $B$ .

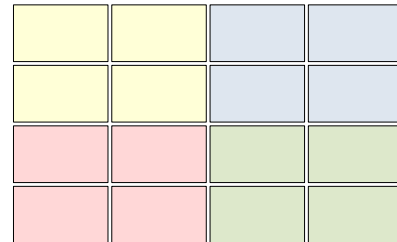


- CUDA permite crear Grids y Bloques bi-dimensionales de hilos que pueden mapearse a la estructura bi-dimensional de las matrices:

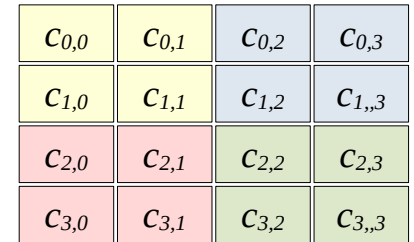
### Ejemplo:



Crear bloques de 2x2 hilos



Crear un grid de 2x2 bloques



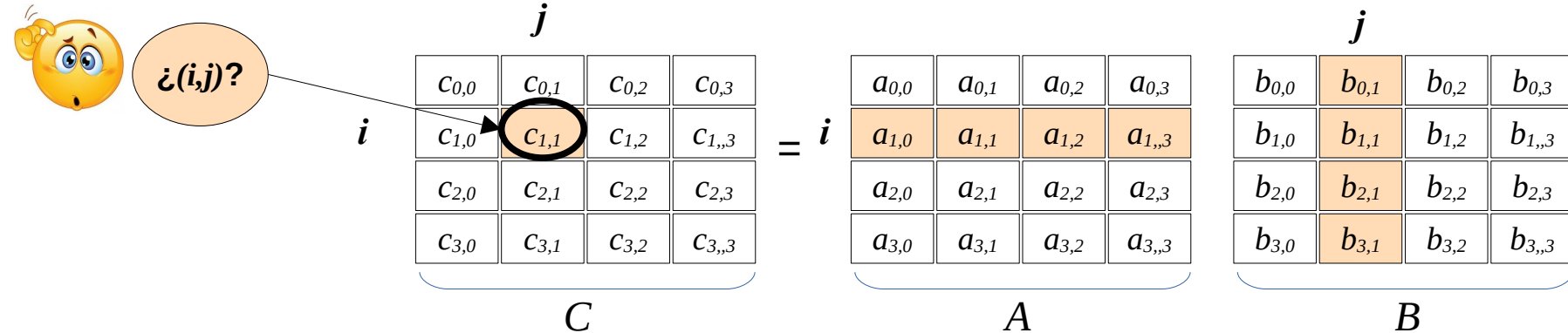
Mapear el grid a la matriz resultado

# Multiplicación de matrices

## Solución en GPU

8

¿Cómo conoce cada hilo CUDA la posición que debe calcular?



Utilizando las variables built-in!!!

Código del **kernel** que calcula la posición  $(i,j)$ :

```
__global__ mm(int *C, int *A, int *B, int N){
    int i = blockIdx.y*blockDim.y + threadIdx.y;
    int j = blockIdx.x*blockDim.x + threadIdx.x;
    int k;
    for( k=0 ; k<N ; k++ )
        C[i*N+j] += A[i*N+k] * B[k+j*N];
}
```



# Multiplicación de matrices

## Solución en GPU

9

Si la dimensión del grid no es proporcional al tamaño de la matriz algunos hilos NO deberían trabajar (más hilos que posiciones a calcular de la matriz C)

Se agrega el siguiente if al código:



```
__global__ mm(int *C, int *A, int *B,int N){
int i = blockIdx.y*blockDim.y + threadIdx.y;
int j = blockIdx.x*blockDim.x + threadIdx.x;
int k;
    if ( i<N && j<N ){
        for( k=0 ; k<N ; k++ )
            C[i*N+j] += A[i*N+k] * B[k+j*N];
    }
}
```



